

DATA-260 Homework 5 Report

Student and Configuration

- Student: Shruti Shetty
- SID4: 9004
- Repository: <https://github.com/shrustishetty-2401/data260-9004>
- Branch: hw5
- PORT_BASE: 8004
- PREFIX: s9004
- SEED: 9004
- VERIFY_SEED: 269004
- DOMAIN_ID: 4
- Local model: qwen3:8b
- Hardware: MacBook Pro, Apple M1 Pro chip, 10-core CPU (8 performance and 2 efficiency cores), 32 GB memory
- Tagged commit (hw5): 4c6984956115f8c50e3f0573313711fba2c50528
- Collaborator access: Sbnikitha and supriyaselvangesan have access to the GitHub repository.

This homework extends the existing DATA-260 HW4 repository. The implementation includes a related package entity, package and vulnerability CRUD APIs, Redux Toolkit state management, two MCP servers, retry handling, a safe tool execution layer, and a local Ollama agent.

Part 1: Database and API Extension

Database Design

The HW5 database adds a `packages` table to the existing vulnerability database. Each package has an integer primary key, name, ecosystem, unique package code, and creation and update timestamps.

The `vulnerabilities` table was extended with:

- `package_id` as a foreign key to `packages`
- `vulnerability_code` as a unique identifier
- `available_count` with a numeric default
- `created_at`
- `updated_at`

The migration preserved the existing HW4 records. Existing package names were converted into package records, and every vulnerability was assigned a package ID and deterministic vulnerability code.

The database was verified with MySQL. It contains the packages, vulnerabilities, related_items, users, and sessions tables. The migration created 253 package records, assigned package IDs to all vulnerabilities, and assigned vulnerability codes without missing values.

Package API

The package router provides:

- POST /api/packages
- GET /api/packages
- GET /api/packages/{package_id}
- PUT /api/packages/{package_id}
- DELETE /api/packages/{package_id}
- GET /api/packages/{package_id}/vulnerabilities

The API validates package names, ecosystems, and unique package codes. Deleting a package that still has related vulnerabilities is prevented with an error response.

Vulnerability API

The vulnerability router retains the HW4 CRUD operations and now includes package relationships and HW5 fields in the response. List and detail requests use eager loading so package and related-item information can be returned together.

The API uses Pydantic validation, authenticated sessions, correct HTTP status codes, and JSON error messages. Package creation returned 201 Created, update returned 200 OK, and deletion returned a successful deletion message.

Redux Toolkit and React

The React frontend uses Axios for HTTP requests and Redux Toolkit for centralized report state. The Redux slice defines asynchronous actions for fetching, creating, updating, and deleting reports.

The React components dispatch Redux actions instead of managing all server data locally. The Home component loads reports into the Redux store, CreateRecord dispatches the create action, UpdateRecord dispatches the update action, and DeleteRecord dispatches the delete action.

The frontend production build completed successfully with Vite.

Part 2: MCP Servers

Meals MCP Server

The Meals MCP server uses the TheMealDB public API through a local STDIO MCP server. The server exposes exactly four tools:

1. search_meals_by_name(query, limit=5)

```
2. meals_by_ingredient(ingredient, limit=12)
3. random_meal()
4. meal_details(id)
```

The server uses HTTP requests with a bounded timeout and returns simplified JSON results. Search results include the meal ID, meal name, area, category, and thumbnail. Detailed meal results include the complete meal information returned by TheMealDB.

The server was tested through MCP Inspector. The search tool successfully returned Spicy Arrabiata Penne for the query Arrabiata. The ingredient search returned chicken-based meals. The random-meal tool returned one meal, and the meal-detail tool returned the detailed information for meal ID 52771.

Domain MCP Server

The domain MCP server uses the local `s9004_re1` database and exposes exactly three read-only tools:

```
1. search_vulnerabilities(query, limit=10)
2. vulnerability_detail(vulnerability_id)
3. vulnerability_aggregate()
```

Each tool returns the same envelope:

```
```json { "ok": true, "data": {}, "error": null }
```

## Part 3: Retry and Fault-Injection Benchmark

The retry layer uses bounded retries with exponential backoff. A failed operation can be attempted up to three times. The delay increases after each retry, which prevents immediate repeated requests from overwhelming a service.

The retry behavior was tested in three required cases:

- First attempt succeeds immediately.
- First attempt fails with a temporary error and the second attempt succeeds.
- All three attempts fail and the operation returns a structured failure.

The benchmark uses `VERIFY_SEED=269004` for deterministic fault injection. Three rejection rates were tested:

Rejection rate	Runs
0%	50
20%	50
50%	50

The benchmark generated 150 raw records in `raw/retry_records.jsonl`. The metrics file records the number of successes, failures, success rate, mean attempts, and p99 elapsed time for each rejection rate.

The raw records preserve each individual request, including its attempts, error messages, retry status, and final result. This makes the benchmark reproducible and allows the reported metrics to be independently checked.

The retry design is useful for batch workloads because a temporary failure can be retried without stopping the entire run. For interactive requests, the retry limit is important because users should not wait indefinitely. The bounded three-attempt design balances reliability with predictable response time.

## Required Retry Metrics

Injected failure rate	Requests	Success rate	Mean latency (ms)	P99 latency (ms)	Mean attempts
0%	50	100%	0.00006	0.002	1.00
20%	50	100%	0.00806	0.132	1.14
50%	50	100%	0.0166	0.058	1.52

The retry policy is suitable for interactive use because all requests completed successfully with a maximum of three attempts and very low latency. For batch processing, I would allow more retries and slightly longer backoff delays because batch jobs can tolerate additional processing time in exchange for improved resilience during temporary failures.

## Domain Tool Input Contracts and Rejected Calls

All three domain tools use the response envelope:

```
```json {"ok": true, "data": {}, "error": null}
```

Part 4: Safe Tool Execution

The `execute_tool(name, inputs)` function is the single entry point for domain-tool execution. It accepts a tool name and input object, validates the request, calls only an approved domain tool, and returns a JSON string.

The approved tools are:

- `search_vulnerabilities`
- `vulnerability_detail`
- `vulnerability_aggregate`

Unknown tools are rejected. Inputs must be JSON objects, and aggregate requests reject unexpected fields. Exceptions are converted into the standard response envelope instead of being exposed as unhandled errors.

The offline test runner verified six cases:

Test	Result
Valid vulnerability search	PASS

Test	Result
Invalid vulnerability search	PASS
Valid vulnerability detail	PASS
Invalid vulnerability detail	PASS
Valid vulnerability aggregate	PASS
Invalid vulnerability aggregate	PASS

The final result was 6/6 tests passed. These tests run without an LLM and directly verify the tool boundary, input validation, error handling, and response-envelope format.

##Part 5: Local Agent and Safety Controls

The local agent uses Ollama with the qwen3:8b model. The model receives a system prompt describing the three approved domain tools and returns either a tool request or a final response.

The agent tracks every step with:

- Step number
- Tool name
- Tool input
- Tool result
- Final response
- Stop reason

The agent stops when the model finishes, when a safety rule blocks the request, or when the maximum step count is reached. All runs are stored in `raw/agent_runs.jsonl`.

The safety rule blocks destructive or sensitive requests containing terms such as `delete`, `drop`, `password`, `secret`, `credential`, and `token`. The request to delete all vulnerability records was blocked before any database tool was called.

The offline tests passed:

- Safety-block test: PASS
- Maximum-step test: PASS

The four live Ollama scenarios were:

Scenario	Stop reason	Steps
Vulnerability search	Maximum step limit	3
Vulnerability detail	Maximum step limit	3
Vulnerability aggregate	Model finished	0
Destructive request	Safety block	0

Agent Scenario Metrics

Scenario	Steps	Tool-call count	Stop reason
Vulnerability search	3	3	Maximum step limit

Scenario	Steps	Tool-call count	Stop reason
Vulnerability detail	3	3	Maximum step limit
Vulnerability aggregate	0	0	Model finished
Destructive request	0	0	Safety block

The model sometimes required multiple turns to correct its tool inputs. For example, one run initially supplied a vulnerability code to the numeric-ID detail tool, received a validation error, and then corrected the request using the numeric ID. This shows why structured tool schemas, validation, and bounded steps are important.

Reflection

One representative run was the vulnerability-detail request. The user asked the agent to show the details for vulnerability ID 10002. The harness first received a tool call in which the identifier was represented as the string "10002". The detail tool rejected that input because the identifier must be numeric. The error was returned through the standard {ok, data, error} envelope, so the agent did not crash and could inspect the failure.

On the next step, the model repeated the same string input, which produced the same validation error. On the third step, the model corrected the input to the integer 10002. The tool then returned the vulnerability record, including its title, package, category, availability count, and timestamps. However, the harness stopped because the configured maximum of three tool steps had been reached. Therefore, the final stop reason was max_steps, even though the corrected tool call succeeded.

This run demonstrates why typed schemas and bounded execution are important. The schema prevented a malformed value from reaching the database, while the step limit prevented the agent from continuing indefinitely. It also showed that the model may need more than one turn to recover from an input-format error. The logged steps make this behavior reproducible and explain exactly why the run stopped.

AI Use

1. I used an AI assistant to help interpret the homework requirements, troubleshoot implementation errors, organize the report, and explain how to run and verify the API, MCP servers, retry tests, and agent tests. I wrote and ran the application code, commands, tests, and screenshots myself.
2. One issue I independently verified was the agent's handling of vulnerability IDs. The model initially sent the ID as a string or used the wrong input field, which was not accepted by the numeric-ID detail tool.
3. I detected the issue by checking the returned error envelope and reviewing the recorded steps in raw/agent_runs.jsonl. The log showed the invalid input, the validation error, and the later successful call using the integer ID.
4. I kept input validation and the standard {ok, data, error} response format in the tool layer. This allowed invalid requests to return structured errors without crashing, while valid corrected requests could continue normally.

Verification

The final verification script records:

- Assignment name
- SID4
- Commit hash
- Model
- Port configuration
- Prefix
- SEED
- VERIFY_SEED
- DOMAIN_ID
- Individual pass/fail checks
- Overall verification status

The generated raw artifacts, documentation files, code files, and metrics are stored under `reports/hw05/`.

Submitted Artifacts

The repository contains the required HW5 files:

- `RUN_LOG.txt`
- `METRICS.md`
- `AI_USE.md`
- `REFLECTION.md`
- `verification.json`
- `raw/retry_records.jsonl`
- `raw/retry_metrics.json`
- `raw/agent_runs.jsonl`
- `raw/agent_metrics.json`

The final verification status is passed. The verification record includes the assignment, SID4, commit hash, model, configuration values, individual checks, and overall status.

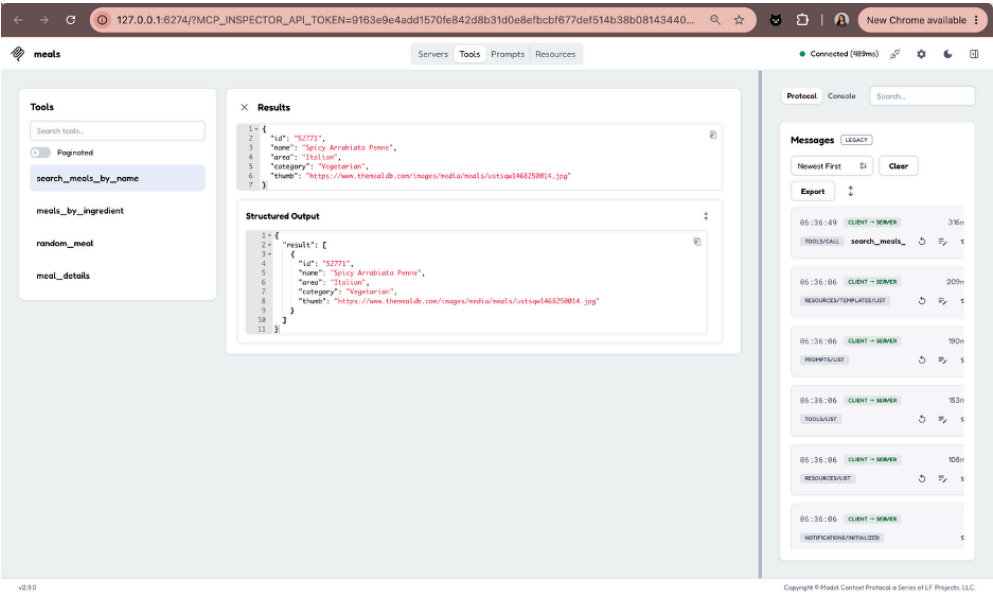
Evidence Screenshots

The `evidence/` directory contains screenshots for:

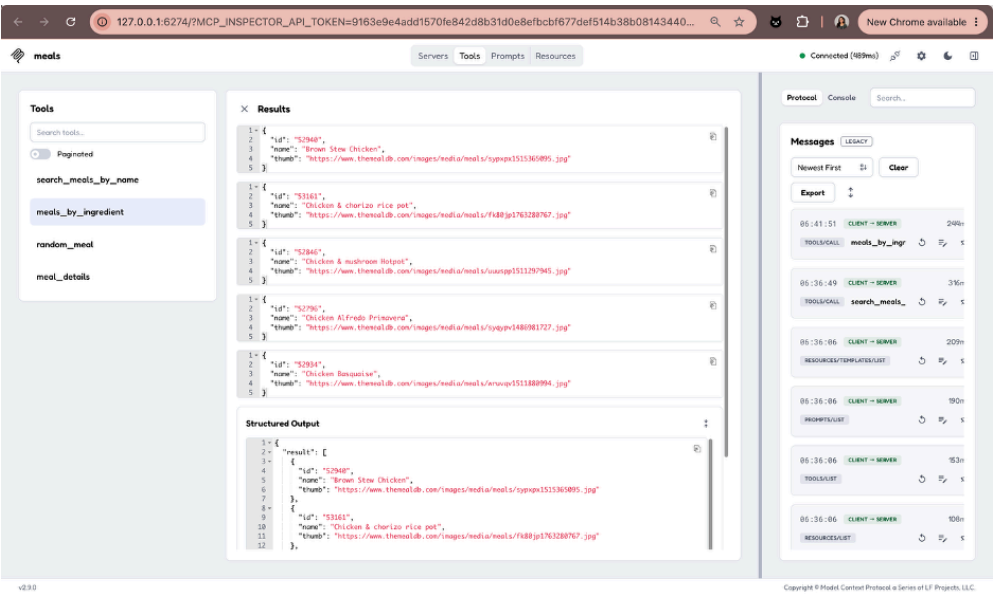
- Package CRUD API responses
- React package and vulnerability pages
- Meals MCP Inspector tools
- Domain MCP Inspector tools
- Retry benchmark output
- Offline tool-runner tests
- Agent safety and maximum-step tests
- Ollama agent scenario results
- Database table verification

Evidence Screenshots

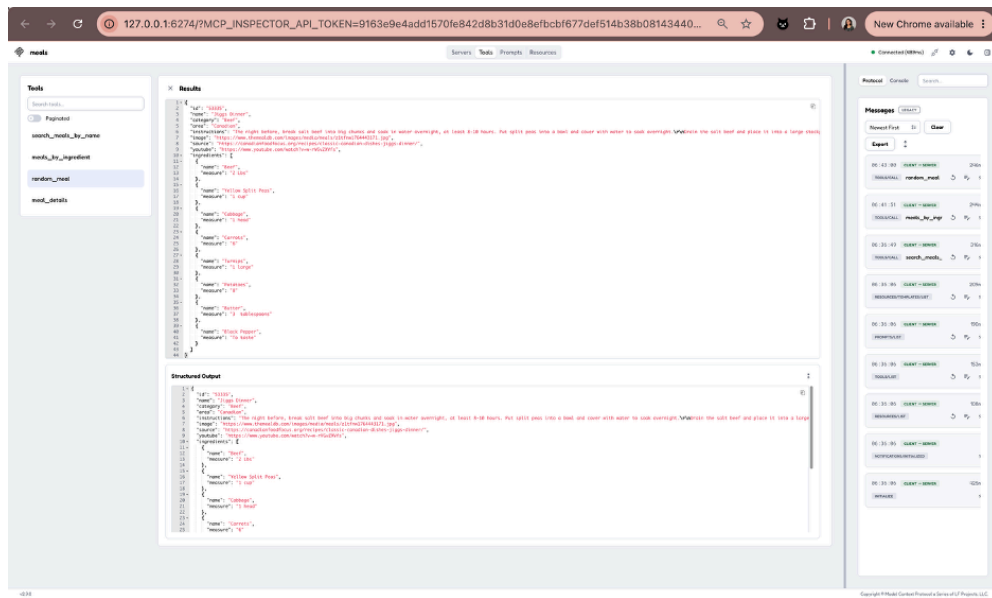
Meals MCP Inspector



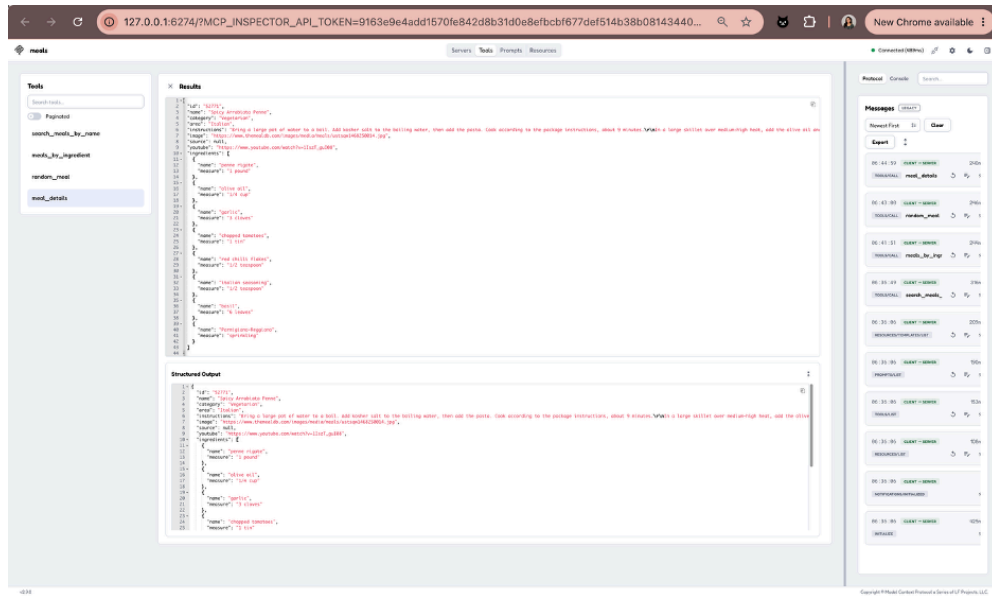
Meals search



Meals ingredient search

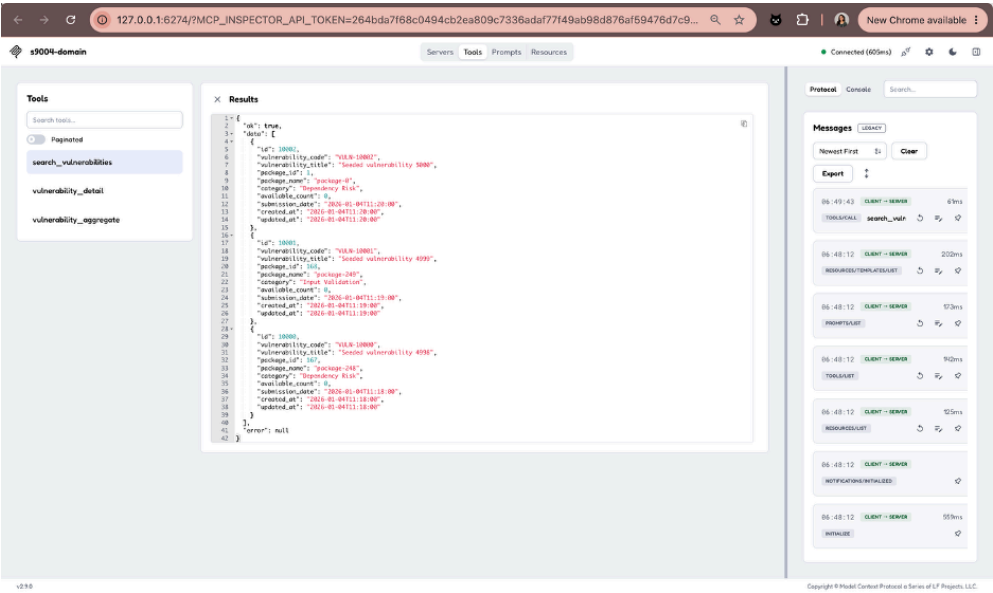


Random meal

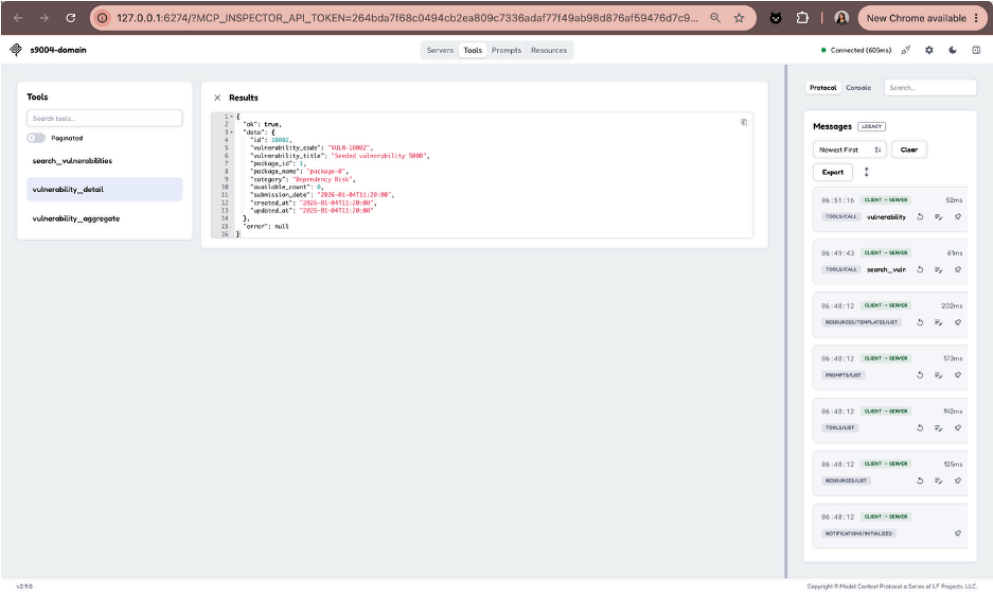


Meal details

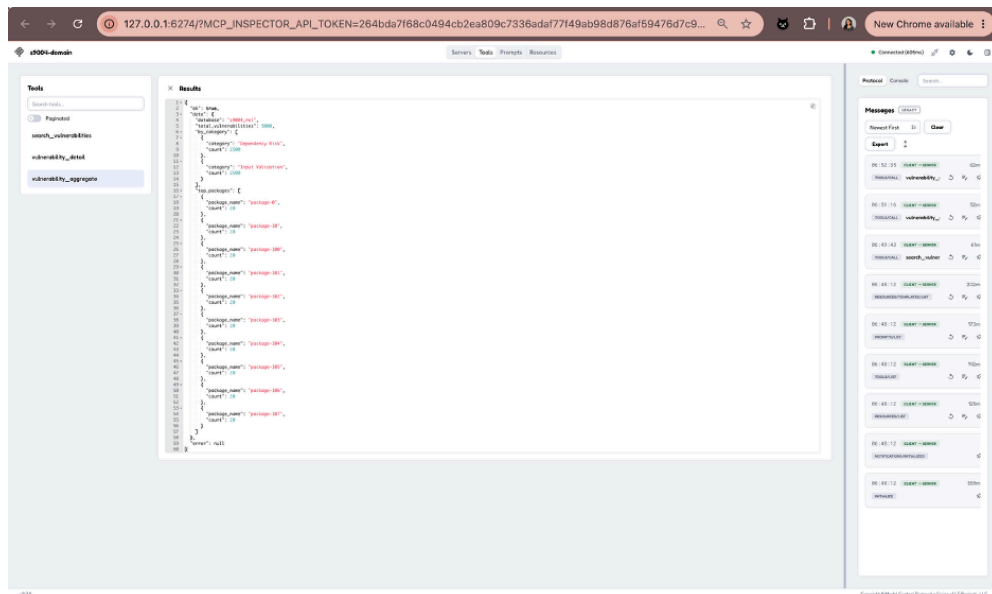
Domain MCP Inspector



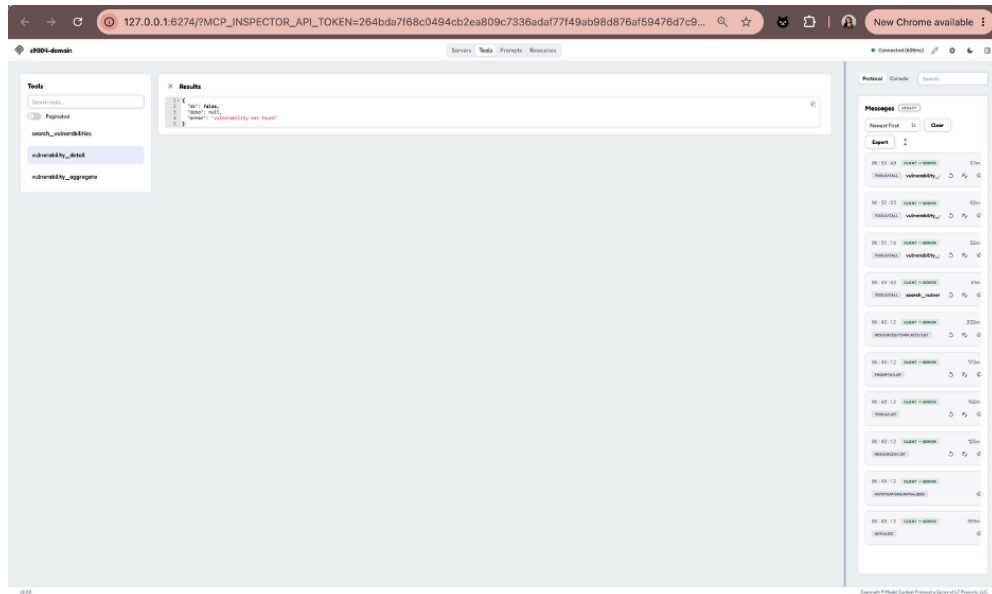
Domain search



Domain detail



Domain aggregate



Invalid domain request

Offline Tests and Verification

```
...0.0.1--port 8004 ... ..004/code --zsh ... ..9004/code --zsh ... ..9004/code --zsh ...004/code --zsh ..004/code --zsh ..004/code --zsh ... ..mcp_server.py ...F6426BF89C96 +

{"ok": true,
 "result": {
   "message": "first try succeeded"
 },
 "attempts": [
   {
     "attempt": 1,
     "status": "success",
     "elapsed_ms": 0.001
   }
 ],
 "total_attempts": 1
 },
 "retry_then_success": {
   "ok": true,
   "result": {
     "message": "second try succeeded"
   },
   "attempts": [
     {
       "attempt": 1,
       "status": "retryable_failure",
       "error": "temporary timeout",
       "elapsed_ms": 0.001
     },
     {
       "attempt": 2,
       "status": "success",
       "elapsed_ms": 0.007
     }
   ],
   "total_attempts": 2
 },
 "all_retries_fail": {
   "ok": false,
   "result": null,
   "attempts": [
     {
       "attempt": 1,
       "status": "retryable_failure",
       "error": "tool rejected request",
       "elapsed_ms": 0.002
     },
     {
       "attempt": 2,
       "status": "retryable_failure",
       "error": "temporary timeout",
       "elapsed_ms": 0.009
     },
     {
       "attempt": 3,
       "status": "retryable_failure",
       "error": "tool rejected request",
       "elapsed_ms": 0.012
     }
   ]
 },
 ],
 }
```

Retry demo

```
...0.0.1--port 8004 ... ..004/code --zsh ... ..9004/code --zsh ... ..9004/code --zsh ...004/code --zsh ..004/code --zsh ..004/code --zsh ... ..mcp_server.py ...F6426BF89C96 +

Last login: Wed Sep 30 23:54:56 on ttys007
(base) srusti@Ashiks-MacBook-Pro code % cd ~/data268-9804/code
conda activate data268
python tool_runner.py
PASS search_valid
PASS search_invalid
PASS detail_valid
PASS detail_invalid
PASS aggregate_valid
PASS aggregate_invalid
4/6 tests passed
(data268) srusti@Ashiks-MacBook-Pro code %
```

Tool runner tests

```

}
}
(data268) srusti@Ashiks-MacBook-Pro code % cd ~/data268-9804/code
conda activate data268
python agent_runner.py
PASS safety_black
PASS max_steps
2/2 agent tests passed
(data268) srusti@Ashiks-MacBook-Pro code %
```

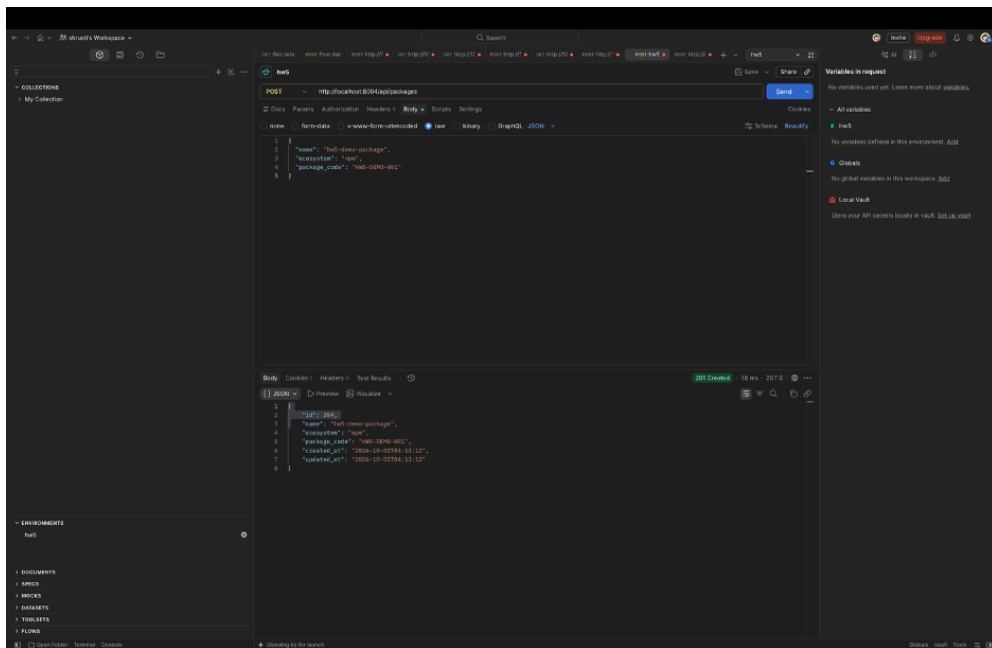
Agent offline tests

```
(data268) struti@Aashiks-MacBook-Pro code % cd ~/data268-9804/code
conda activate data268
python agent_runner.py
PASS safety_block
PASS max_steps
2/2 agent tests passed
(data268) struti@Aashiks-MacBook-Pro code % cd ~/data268-9804/code
conda activate data268
python verify_hw5.py
{
  "assignment": "DATA-268 HW5",
  "sid": "9804",
  "commit_hash": "9802b0c62d974f5c8882322b59869d1fca75dc4c",
  "model": "qwen3:8b",
  "port_base": 8080,
  "prefix": "v9804",
  "seed": 9804,
  "verify_seed": 269804,
  "domain_id": 4,
  "checks": {
    "webui_server_present": true,
    "domain_mcp_server_present": true,
    "retry_demo_present": true,
    "retry_benchmark_present": true,
    "tool_runner_present": true,
    "agent_runner_present": true,
    "agent_domain_present": true,
    "raw_retry_records_present": true,
    "raw_retry_records_have_100_lines": true,
    "retry_metrics_present": true,
    "agent_runs_present": true,
    "agent_metrics_present": true,
    "reflection_present": true,
    "run_log_present": true,
    "metric_document_present": true,
    "ai_use_present": true,
    "tool_runner_metrics_valid": true,
    "retry_metrics_valid": true,
    "agent_metrics_valid": true
  },
  "status": "passed"
}
(data268) struti@Aashiks-MacBook-Pro code %
```

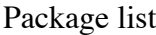
HW5 verification

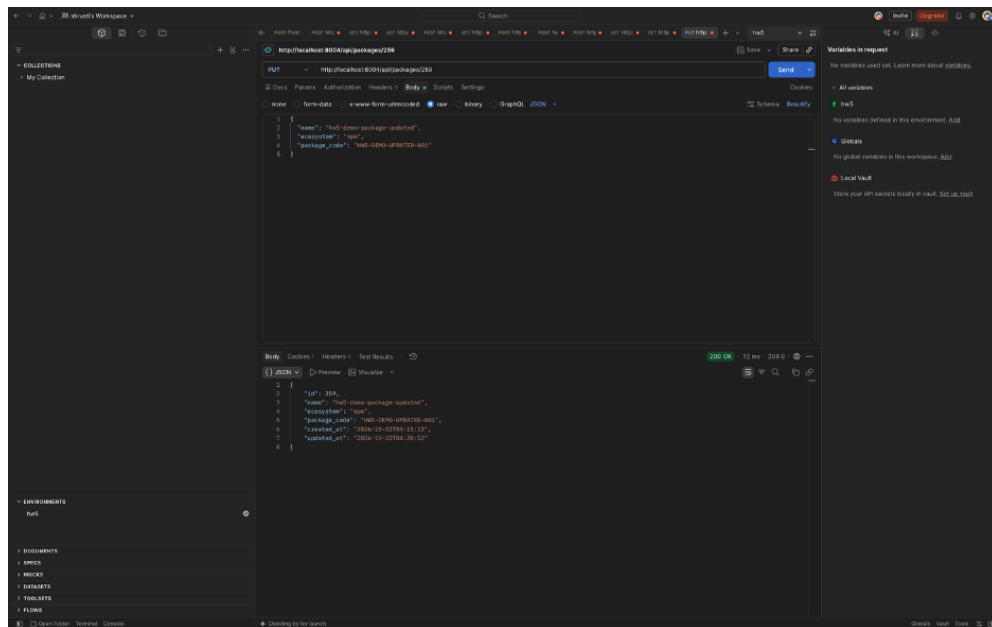
Part 1 API and Database Evidence

Package API

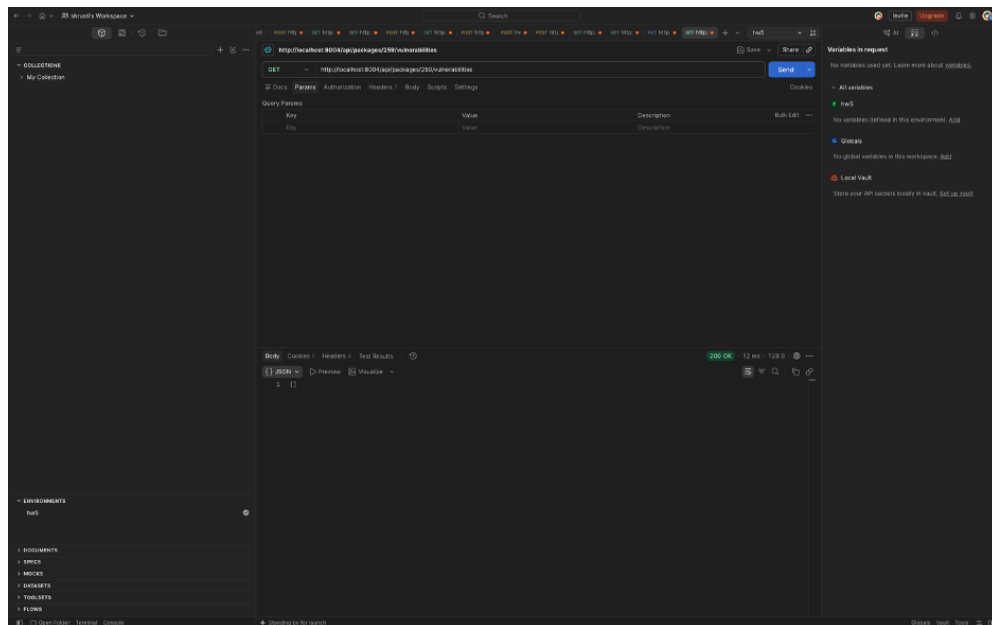


Package create





Package update

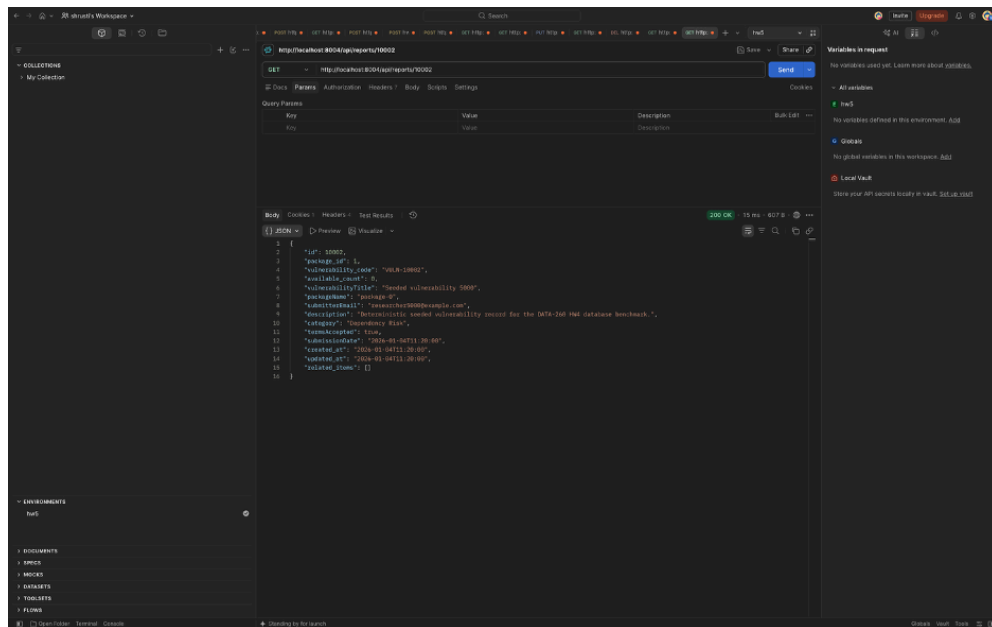


Package vulnerabilities

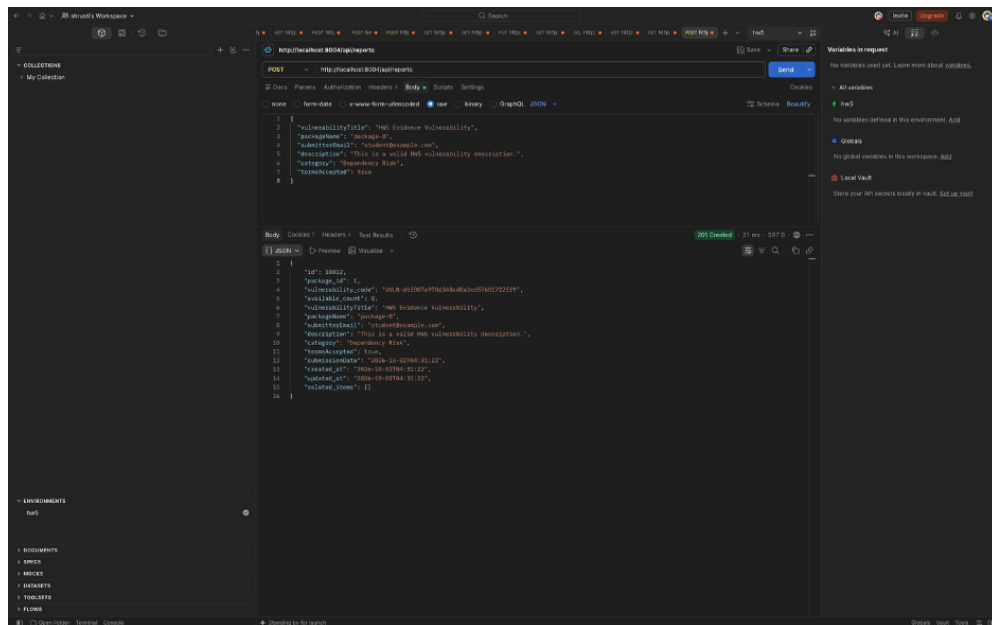


The screenshot shows the Burp Suite interface with the HTTP history tab selected. The selected item is a GET request to `http://localhost:8084/test/reports`. The 'Raw' tab is active, displaying the raw HTTP request. The request is a GET to `http://localhost:8084/test/reports` with a 'Host' header of `localhost:8084`. The body contains a JSON array of two vulnerability records. The first record is for 'SQL-DMO2' with a severity of 'HIGH' and a description of a 'Database user vulnerability record for the SQL-DMO2 web database benchmark'. The second record is for 'SQL-DMO2' with a severity of 'HIGH' and a description of a 'Database user vulnerability record for the SQL-DMO2 web database benchmark'.

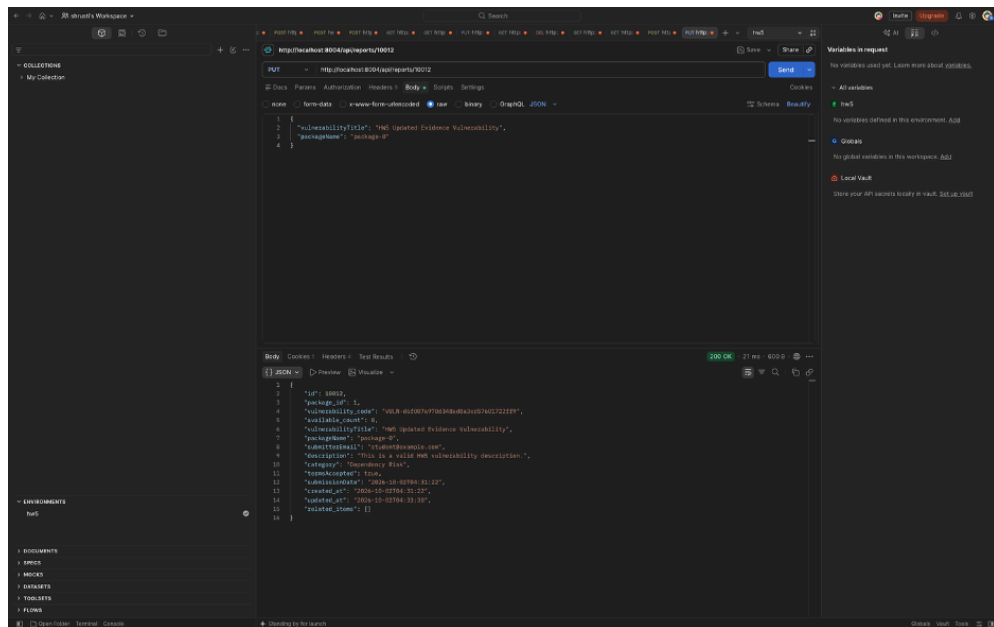
Report list



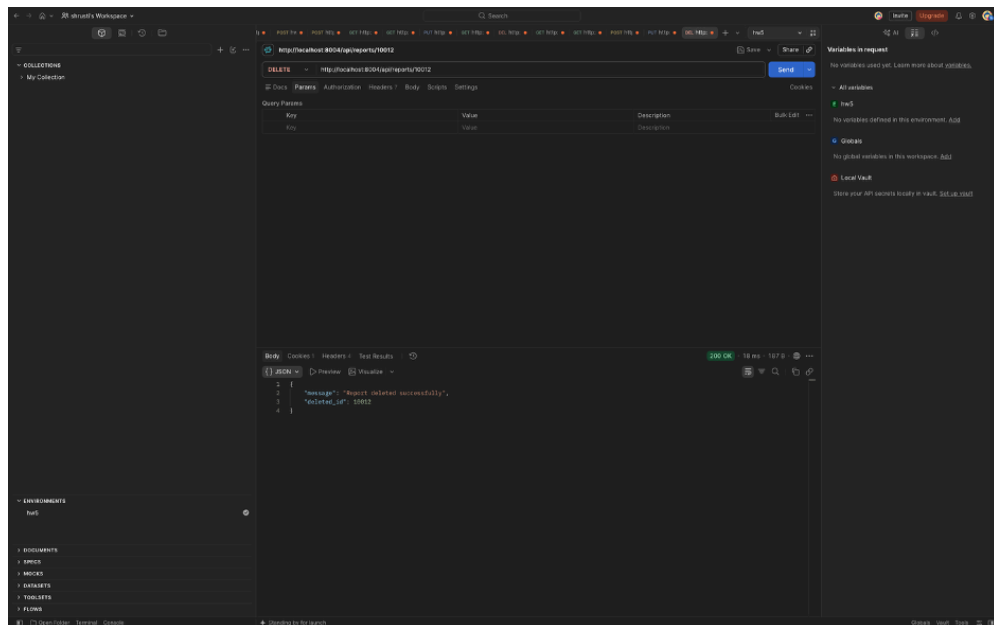
Report detail



Report create

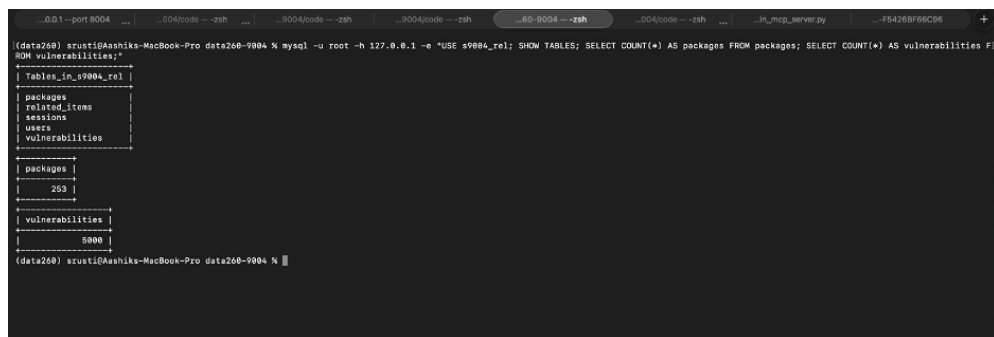


Report update



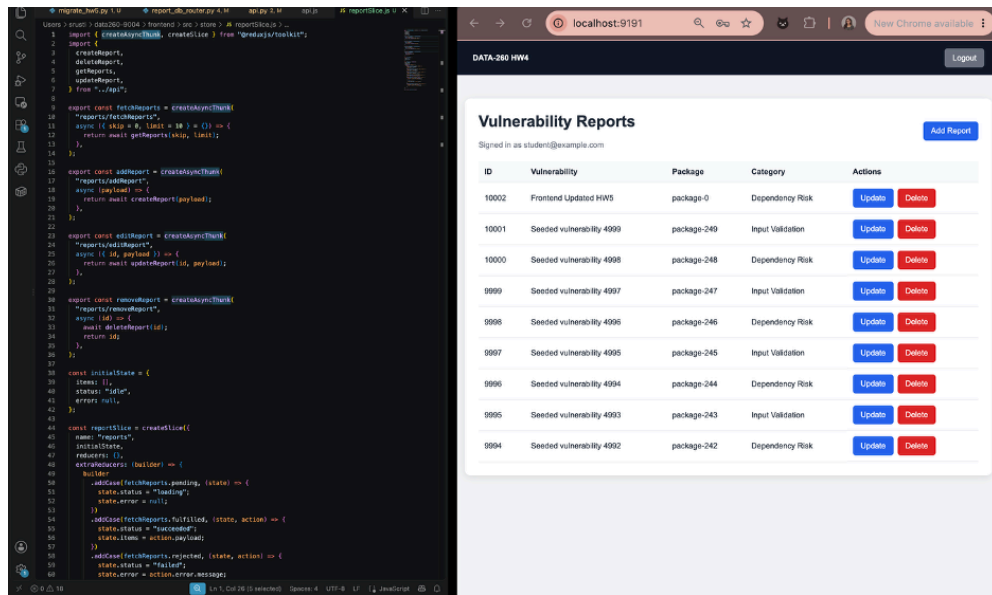
Report delete

Database

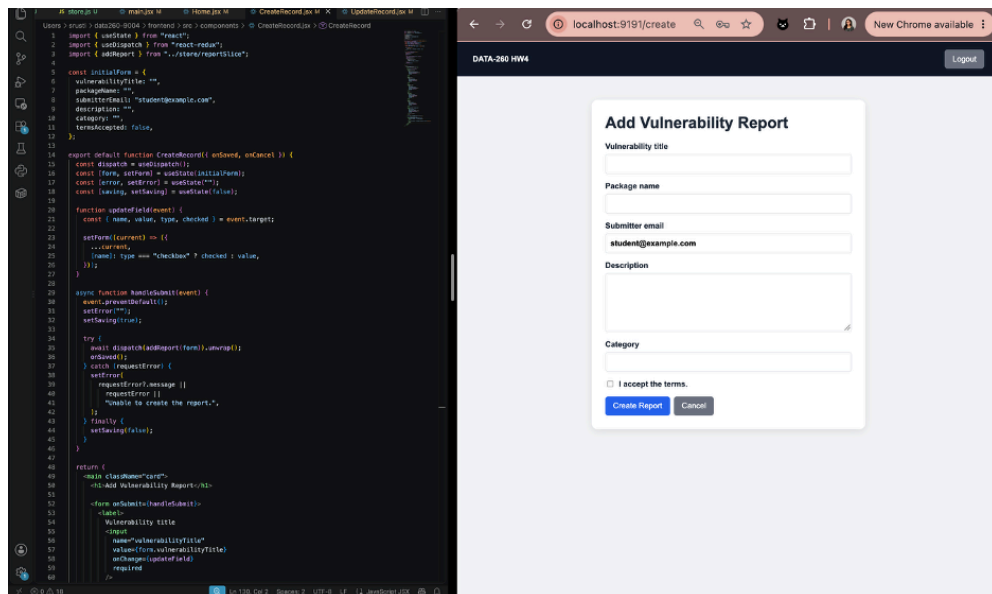


HW5 database tables and counts

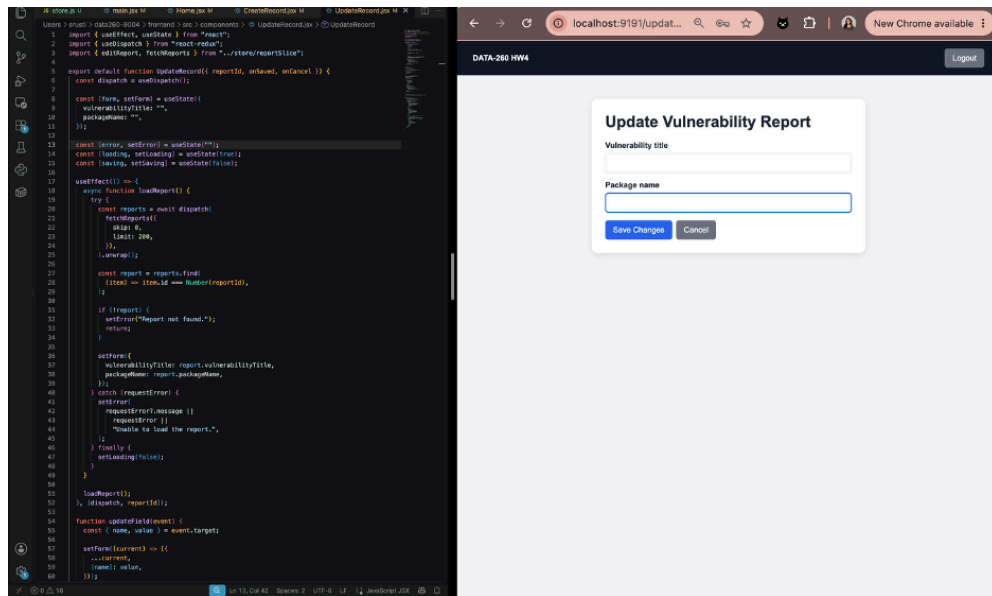
Redux Client Evidence



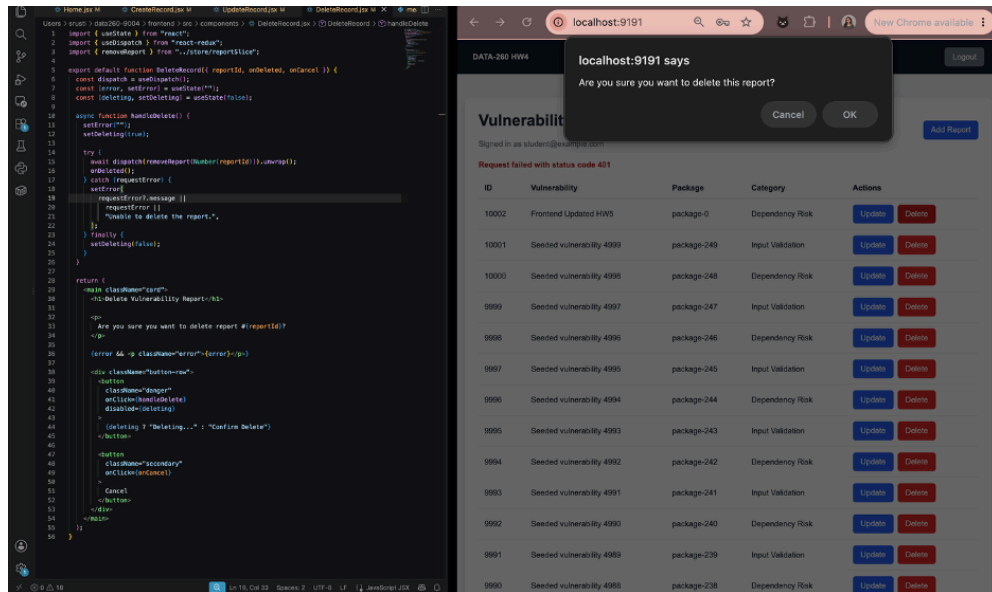
Home page with Redux code



Create page with Redux code



Update page with Redux code



Delete page with Redux code